

API REFERENCE — Pagamenti

Backend Gestionale B&B Cosenza Versione API v1 · Documento aggiornato al 20 settembre 2026 · Copertura: Step G

Come usare questo documento

Segue la stessa struttura del documento del modulo Booking, con cui va letto in parallelo: qui c'è il pagamento, là il ciclo di vita della prenotazione che il pagamento conferma.

Questo documento viene aggiornato a ogni modifica dell'area. Il *Changelog* in fondo traccia cosa è cambiato e quando.

1. L'idea portante: autorizzare non è incassare

È la scelta da cui discende tutto il resto, e capirla prima risparmia la lettura di metà documento.

Una carta si può **autorizzare** senza **incassare**. L'autorizzazione blocca l'importo sulla carta ma non lo preleva; il prelievo avviene con una seconda chiamata, che parte **dal nostro backend**.

Il flusso è quindi:

#	Chi	Cosa
1	SPA	chiede al backend di avviare il pagamento
2	Backend	crea un Payment Intent a incasso differito e restituisce il client_secret
3	Browser ↔ Stripe	l'ospite paga; qui avviene anche l'autenticazione della banca (SCA)
4	Stripe → Backend	notifica amount_capturable_updated : l'importo è pronto, non ancora prelevato
5	Backend	verifica: la camera è ancora dell'ospite?
6a	Backend	sì → incassa → prenotazione CONFIRMED → email di conferma
6b	Backend	no → rilascia l'autorizzazione → nessun addebito → email di scuse

Perché non si incassa subito

Fra il momento in cui l'ospite paga e quello in cui lo scopriamo passa del tempo, e non dipende da come è fatto il sito:

- **SCA**: l'ospite viene mandato all'app della banca. Cinque minuti sono normali, e spesso il primo tentativo fallisce.
- **Ritentativi**: carta rifiutata, l'ospite ne cerca un'altra.
- **Scheda lasciata aperta**: apre il pagamento, lo interrompe, torna dopo.
- **Notifica in ritardo**: se il nostro server è irraggiungibile quando Stripe ci avvisa — un deploy, per esempio — Stripe ritenta a intervalli crescenti **fino a tre giorni**. Questa è la causa decisiva, e non ha nulla a che fare con il comportamento dell'ospite.

Con l'incasso immediato, tutti questi casi possono finire con i soldi dell'ospite presi e la camera venduta a un altro. Con l'incasso differito quel caso **non esiste**: al suo posto c'è "autorizzazione rilasciata", che non muove denaro e non

richiede rimborsi.

C'è anche un effetto collaterale prezioso: se il nostro server non elabora mai la notifica, l'autorizzazione **scade da sola** dopo qualche giorno e l'ospite non viene addebitato. Un guasto silenzioso produce un non-addebito invece di un addebito da rimborsare.

Cosa costa

L'ospite vede un blocco sulla carta. Se rilasciamo, l'importo risulta "in sospeso" nel suo estratto conto e sparisce in qualche giorno, a seconda della banca. L'email glielo dice esplicitamente.

La SPA ha uno stato in più. Dopo l'autorizzazione il browser non può dire "confermato": deve mostrare un *"stiamo confermando la prenotazione..."* per il secondo o due che serve al backend per incassare, e poi verificare l'esito.

2. PCI-DSS

Il numero di carta non tocca mai questo backend. Il browser lo invia direttamente a Stripe tramite Stripe.js / Payment Element; noi maneggiamo solo identificativi opachi (`pi_...` , `client_secret`).

Dopo l'incasso conserviamo **circuito e ultime quattro cifre** (`card_brand` , `card_last4`), che servono al back-office per riconoscere un pagamento e non sono dati di autenticazione.

Non conserviamo mai: PAN completo, CVV, banda magnetica, PIN. Non è una promessa: sono dati che non arrivano mai fino a qui.

Ambito di conformità: **SAQ-A-EP**, perché la pagina di pagamento è servita da noi anche se i campi della carta sono iframe di Stripe.

3. Configurazione

Variabile	Default	Note
<code>STRIPE_ENABLED</code>	<code>false</code>	Con <code>false</code> l'endpoint <code>/intent</code> risponde <code>502</code>
<code>STRIPE_SECRET_KEY</code>	—	Chiave server. Mai nel repository
<code>STRIPE_WEBHOOK_SECRET</code>	—	Segreto di firma, diverso dalla chiave server
<code>STRIPE_AUTO_REFUND_ON_CANCELLATION</code>	<code>false</code>	Rimborso automatico sugli annullamenti dopo la conferma
<code>STRIPE_WEBHOOK_TOLERANCE_SECONDS</code>	<code>300</code>	Età massima di una firma accettata
<code>BOOKING_PAYMENT_HOLD_MINUTES</code>	<code>30</code>	Blocco dello slot durante il pagamento

`BOOKING_PAYMENT_HOLD_MINUTES` è più lungo di `BOOKING_HOLD_MINUTES` perché misura una cosa diversa: quindici minuti bastano per un clic su un link, non per un pagamento. Allungarlo non apre falle — ciò che tiene davvero lo slot non è questo timer ma l'autorizzazione viva su Stripe.

4. Endpoint

4.1 Prospetto

#	Metodo	Rotta	Accesso	Rate limit
1	POST	/api/v1/payments/intent	Pubblico (codice + email)	10/ora per IP
2	POST	/api/v1/payments/webhook	Firma Stripe	nessuno

1 • POST /api/v1/payments/intent

Avvia il pagamento di una prenotazione.

Accesso	Pubblico, con codice e email
Rate limit	10/ora per IP
Body	PaymentIntentRequestSchema
Risposta	PaymentIntentSchema

Codici: 200 • 402 prenotazione da saldare in struttura • 404 codice o email errati • 409 già confermata • 410 blocco scaduto e slot perduto • 429 • 502 gestore non raggiungibile o pagamenti disattivati

Logica. Verifica codice ed email, controlla che la prenotazione sia **PAY_NOW** e in attesa di pagamento, **proroga il blocco** a **BOOKING_PAYMENT_HOLD_MINUTES**, poi crea il Payment Intent a incasso differito e lo associa alla prenotazione.

L'importo non viene mai accettato dal client. È letto dalla prenotazione, che a sua volta lo ha ricavato da un preventivo firmato e ricalcolato dal server.

Ripetibile senza conseguenze. Se la prenotazione ha già un Payment Intent viene recuperato, non ricreato; e la creazione usa il codice prenotazione come chiave di idempotenza. Due clic su "Paga" non producono due addebiti.

Tre transazioni, mai una sola. La chiamata a Stripe sta *fra* due transazioni del database, mai dentro: un timeout del gestore, dentro una transazione, farebbe rollback della prenotazione — oppure lascerebbe dietro un Payment Intent pagabile senza nulla che gli corrisponda.

Se il blocco era scaduto e lo slot è stato nel frattempo venduto, la riattivazione delle righe camera fallisce contro l'exclusion constraint e la risposta è 409.

Test: test_l_avvio_restituisce_il_segreto_e_l_importo ,
test_l_importo_comunicato_a_stripe_e_quello_della_prenotazione ,
test_avviare_due_volte_non_crea_due_pagamenti , test_l_avvio_proroga_il_blocco ,
test_email_sbagliata_risponde_404 ,
test_una_prenotazione_da_saldare_in_struttura_non_si_paga_online .

```
curl -i -X POST http://localhost:8000/api/v1/payments/intent \  
-H "Content-Type: application/json" \  
-d '{"code":"BB-2026-A7K3QX","email":"mario@example.com"}'
```

```
{
  "client_secret": "pi_3abc_secret_xyz",
  "amount": "180.00",
  "currency": "EUR",
  "booking_code": "BB-2026-A7K3QX"
}
```

2 • POST /api/v1/payments/webhook

Riceve le notifiche di Stripe.

Accesso	Solo Stripe, per firma
Rate limit	nessuno
Body	JSON grezzo di Stripe
Risposta	<code>WebhookResultSchema</code>

Codici: `200` • `400` firma non valida • `5xx` elaborazione fallita (Stripe ritenta)

Non compare in Swagger (`include_in_schema=False`): non è una API pubblica.

La firma è l'unica autenticazione. Chiunque conosca l'URL può chiamarlo, solo Stripe sa firmarlo. Per questo non c'è rate limiting: bloccare le notifiche significherebbe perdere pagamenti, e chi non ha la firma viene comunque respinto.

⚠ Il corpo viene letto grezzo. L'endpoint non dichiara un modello Pydantic per il payload. Se FastAPI deserializzasse il JSON e la libreria lo riserializzasse per verificare la firma, basterebbe un ordine di chiavi diverso o uno spazio in più perché il confronto fallisse. È l'errore classico di questa integrazione, e produce un endpoint che rifiuta **tutte** le notifiche legittime.

Eventi gestiti

Evento	Effetto
<code>payment_intent.amount_capturable_updated</code>	Verifica e incasso , oppure rilascio. È il punto in cui si decide
<code>payment_intent.succeeded</code>	Rete di sicurezza: completa se l'incasso era avvenuto ma la scrittura no
<code>payment_intent.payment_failed</code>	<code>payment_status = FAILED</code> , prenotazione ritentabile
<code>payment_intent.canceled</code>	Registrato nei log
<code>charge.refunded</code>	<code>payment_status = REFUNDED</code>
tutti gli altri	<code>200</code> e una riga di log

Rispondere con un errore a un evento che non ci riguarda farebbe ritentare Stripe per giorni senza alcun motivo.

Idempotenza. Stripe consegna *at-least-once* e ritenta finché non riceve `200`. Ogni evento viene registrato in `stripe_events` con `INSERT ... ON CONFLICT DO UPDATE ... WHERE processed_at IS NULL`: un evento già **elaborato** viene scartato come duplicato, uno rimasto **a metà** viene ripreso. La distinzione conta: un semplice `DO NOTHING` renderebbe permanente ogni guasto a metà strada.

Gli effetti a valle sono comunque idempotenti di loro — la prenotazione viene bloccata con `FOR UPDATE` e la conferma non fa nulla se lo stato è già `CONFIRMED`. Il registro riduce il lavoro inutile; la correttezza non dipende solo da lui.

Un errore deve restare un errore. Rispondere `200` a una notifica non elaborata direbbe a Stripe di non riprovare, e quel pagamento sarebbe perso per sempre. Le eccezioni risalgono all'handler globale, che risponde `5xx`.

Esiti possibili (campo `outcome`): `CONFIRMED` · `ALREADY_CONFIRMED` · `SLOT_LOST` · `AMOUNT_MISMATCH` · `PAYMENT_FAILED` · `REFUNDED` · `CANCELED` · `DUPLICATE` · `IGNORED` · `UNKNOWN_BOOKING`

Test: `test_ciclo_completo_dal_pagamento_alla_conferma`, `test_lo_slot_perduto_non_produce_addebito`, `test_lo_stesso_evento_due_volte_ha_un_solo_effetto`, `test_una_firma_non_valida_non_produce_effetti`, `test_un_importo_diverso_dal_dovuto_non_viene_incassato`, `test_un_pagamento_rifiutato_resto_ritentabile`, e altri sei.

5. Verifiche prima dell'incasso

Tre controlli, nell'ordine in cui vengono eseguiti. Il primo che fallisce impedisce l'incasso.

5.1 La prenotazione esiste

Un'autorizzazione senza prenotazione viene **rilasciata**: non deve restare viva.

5.2 L'importo corrisponde

Si confronta l'importo autorizzato con il totale della prenotazione. Vale in entrambe le direzioni: non si incassa né meno né più del dovuto. Un disallineamento è un segnale di manomissione o di una modifica del totale avvenuta dopo la creazione del Payment Intent.

La conversione euro → centesimi avviene su `Decimal`, **mai** su `float`: un arrotondamento qui non è un disallineamento contabile da correggere, è un addebito sbagliato sulla carta di una persona. Le valute senza decimali (yen, won) vengono rifiutate esplicitamente invece di essere convertite male.

5.3 Lo slot è ancora nostro

Non si interroga una query: si **prova a riprendere** le righe camera e si lascia rispondere l'exclusion constraint del database. Se qualcun altro ha prenotato quelle date, l'inserimento viene rifiutato e la risposta è inequivocabile.

È la stessa tecnica dell'anti-overbooking: chiedere "è libero?" e poi agire lascia una finestra fra la domanda e la risposta; agire e lasciar rifiutare il database no.

6. Lo sweeper e le autorizzazioni

Lo sweeper delle scadenze (Step F) ha una regola in più da quando esistono i pagamenti: **non libera mai uno slot su cui c'è un'autorizzazione viva**.

Per ogni prenotazione in attesa di pagamento con blocco scaduto:

Esito del rilascio	Cosa succede
Rilasciata	Lo slot torna in vendita normalmente
Stripe dice "già incassata"	L'ospite ha pagato nell'istante esatto: si conferma la prenotazione
Gestore irraggiungibile	Non si libera nulla, si riprova al giro dopo

L'ultimo caso è la scelta prudente: uno slot invenduto per un'ora costa una notte, un incasso senza camera costa molto di più.

7. Schemi

PAYMENTINTENTREQUESTSCHEMA

Campo	Tipo	Vincoli
code	string	max 20, normalizzato in maiuscolo
email	EmailStr	obbligatoria

Richiede entrambi, come il lookup: il solo codice non deve bastare ad aprire il pagamento di una prenotazione altrui.

PAYMENTINTENTSCHEMA

Campo	Tipo	Note
client_secret	string · null	Da passare a Stripe.js. Consente di pagare quella prenotazione e nient'altro
amount	Decimal	Totale, da mostrare all'ospite
currency	string	
booking_code	string	

WEBHOOKRESULTSCHEMA

Campo	Tipo	Note
event_id	string	
event_type	string	
outcome	string	Quale ramo è stato percorso

8. Copertura dei test

File	Test	DB	Cosa verifica
test_payment_service.py	17	no	Conversione importi, macchina a stati del gateway, firma
test_payment_api.py	16	sì	Ciclo completo, slot perduto, idempotenza, firma, importo

Il test che conta più di tutti

`test_lo_slot_perduto_non_produce_addebito`. L'ospite paga, ma le camere sono state vendute a un altro. Il test verifica tre cose insieme: **nessun incasso**, **nessun rimborso**, e un'email che dice all'ospite che non gli è stato addebitato nulla.

È la ragione per cui esiste l'incasso differito. Con l'incasso immediato quel test non si potrebbe nemmeno scrivere: si potrebbe solo verificare che il rimborso sia partito.

9. Checklist di verifica manuale

Con le chiavi di test di Stripe e `stripe listen --forward-to localhost:8000/api/v1/payments/webhook`.

Percorso felice

- [] `POST /intent` restituisce `client_secret` e l'importo scontato del 10%
- [] Pagando con `4242 4242 4242 4242` la prenotazione passa a `CONFIRMED`
- [] Arriva l'email di conferma con il link di gestione
- [] Nel cruscotto Stripe l'importo risulta **incassato**, non solo autorizzato
- [] `card_brand` e `card_last4` sono valorizzati; nessun PAN in tabella o nei log

Casi limite

- [] Chiamare `/intent` due volte → stesso `client_secret`
- [] Carta `4000 0000 0000 0002` (rifiutata) → prenotazione ancora `PENDING_PAYMENT`, ritentabile
- [] Carta `4000 0025 0000 3155` (richiede SCA) → il flusso si completa dopo l'autenticazione
- [] `POST /webhook` con firma inventata → `400`, nessun effetto
- [] Rieseguire la stessa notifica → `DUPLICATE`, nessuna seconda email
- [] Prenotazione `PAY_ON_ARRIVAL` su `/intent` → `402`

Slot perduto

- [] Autorizzare un pagamento, poi liberare a mano le righe camera e prenotare le stesse date da un altro
- [] Inoltrare la notifica → esito `SLOT_LOST`
- [] **Nel cruscotto Stripe l'autorizzazione risulta rilasciata, non incassata e non rimborsata**
- [] L'ospite riceve l'email che dice che non è stato addebitato nulla

10. Changelog

Data	Step	Modifiche
20/09/2026	G	Payment Intent a incasso differito · webhook firmato e idempotente · verifica dell'importo e dello slot prima dell'incasso · rilascio dell'autorizzazione se lo slot è perduto · sweeper che annulla le autorizzazioni prima di liberare · email <code>booking_slot_lost</code>

11. Da completare

Contenuto	Note
Rimborso sulla cancellazione dopo la conferma	<code>STRIPE_AUTO_REFUND_ON_CANCELLATION</code> esiste ma non è ancora collegato al percorso di annullamento
Penale parziale sul rimborso	Oggi il rimborso è totale o niente
Riconciliazione periodica	Confrontare <code>stripe_events</code> non elaborati con lo stato delle prenotazioni

Documento generato automaticamente dal sorgente `docs/api/payment_api.md`. Per segnalare una discrepanza fra documento e codice, fa fede il codice.